

Universitatea de Vest din Timișoara

Facultatea de Matematică și Informatică

Specializarea: Informatică în limba română

Analiza comparativă a algoritmilor de sortare

Evaluare teoretică și experimentală în limbajul Python

Student: Minciuna David-Florian

Email: david.minciuna06@e-uvt.ro

Grupa: 4

Disciplina: Scriere academică

Prof. coordonator: Adrian Crăciun

An universitar 2025–2026

Mai 2026

Timișoara, România

Rezumat

This paper presents a comparative analysis of several classical sorting algorithms implemented in the Python programming language. Both the theoretical and practical performances of Bubble Sort, Selection Sort, Insertion Sort, and Quick Sort are studied and evaluated. The main objective of the project is to highlight the efficiency differences between these algorithms and to identify the situations in which each sorting method can be used effectively. The paper also analyzes the influence of input data structure on algorithm performance and execution time. Experimental tests were performed on multiple types of datasets, including random, sorted, reverse sorted, and nearly sorted lists. The obtained results demonstrate that Quick Sort provides the best overall performance in most practical situations, while simpler algorithms become inefficient for large datasets.

Cuvinte cheie: algoritmi de sortare, Python, complexitate, Quick Sort, evaluare experimentală, performanță.

Cuprins

1	Introducere	4
1.1	Motivarea problemei	4
1.2	Soluții existente	4
1.3	Limitările soluțiilor analizate	5
1.4	Obiectivele proiectului	5
1.5	Organizarea lucrării	5
2	Prezentarea formală a problemei	5
2.1	Definirea problemei de sortare	5
2.2	Proprietăți urmărite	6
3	Algoritmii analizați	6
3.1	Bubble Sort	6
3.2	Selection Sort	6
3.3	Insertion Sort	7
3.4	Quick Sort	7
4	Analiza complexității	7
5	Modelarea și implementarea soluției	8
5.1	Arhitectura generală a programului	8
5.2	Generarea datelor de test	8
5.3	Mediul experimental	8
5.4	Exemplu de implementare Quick Sort	9
5.5	Măsurarea timpului de execuție	9
6	Studii de caz și rezultate experimentale	10
6.1	Descrierea testelor	10
6.2	Grafic comparativ	10
6.3	Interpretarea rezultatelor	11
7	Comparație cu literatura de specialitate	11
8	Avantaje și dezavantaje	12
9	Limitări ale studiului	12
10	Concluzii și direcții viitoare	13

11 Bibliografie	14
A Anexe	14
A.1 Observații tehnice	14

1. Introducere

Sortarea reprezintă procesul de aranjare a elementelor unei colecții într-o anumită ordine, de obicei crescătoare sau descrescătoare. În informatică, această colecție poate fi o listă, un vector, un fișier de date sau o structură mai complexă, iar elementele pot fi numere, șiruri de caractere sau obiecte care pot fi comparate după un anumit criteriu.

În cadrul acestui proiect, sortarea se referă la ordonarea crescătoare a unei liste de numere întregi. Pentru rezolvarea acestei probleme sunt utilizați patru algoritmi clasici: Bubble Sort, Selection Sort, Insertion Sort și Quick Sort. Fiecare algoritm aplică o strategie diferită, iar diferențele dintre ele devin vizibile mai ales atunci când dimensiunea listei crește sau când datele au o anumită structură inițială.

Studierea algoritmilor de sortare este importantă deoarece aceștia formează o bază pentru înțelegerea eficienței algoritmice. Prin analiza lor pot fi observate concepte fundamentale precum complexitatea temporală, complexitatea spațială, cazurile favorabile și nefavorabile, precum și impactul datelor de intrare asupra performanței.

1.1. Motivarea problemei

Sortarea este una dintre cele mai frecvent întâlnite operații din informatică. Ea apare în organizarea bazelor de date, în afișarea rezultatelor căutărilor, în analiza statistică, în prelucrarea datelor și în numeroase aplicații software care au nevoie de informații ordonate pentru a funcționa eficient.

Deși la prima vedere sortarea poate părea o problemă simplă, alegerea algoritmului folosit are o importanță majoră. Pentru liste mici, diferențele dintre algoritmi pot fi greu de observat, însă pentru volume mari de date acestea devin semnificative. Un algoritm lent poate transforma o operație aparent simplă într-un proces costisitor, în timp ce un algoritm eficient poate reduce considerabil timpul de execuție.

În acest proiect sunt analizați patru algoritmi de sortare reprezentativi: Bubble Sort, Selection Sort, Insertion Sort și Quick Sort. Primii trei sunt importanți mai ales din punct de vedere educațional, deoarece ajută la înțelegerea conceptelor de bază, precum comparația, interschimbarea și parcurgerea repetată a unei liste. Quick Sort este inclus deoarece reprezintă o metodă mai performantă, utilizată frecvent în practică datorită comportamentului său eficient în cazul mediu.

1.2. Soluții existente

De-a lungul timpului au fost dezvoltate numeroase metode de sortare, fiecare având avantaje și limitări. Algoritmii bazați pe comparații, precum Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Heap Sort și Quick Sort, ordonează elementele prin compararea directă a valorilor. Există și algoritmi care nu se bazează exclusiv pe comparații, precum Counting Sort sau Radix Sort, însă aceștia sunt eficienți doar în anumite condiții, de exemplu atunci când valorile aparțin unui domeniu restrâns.

Bubble Sort este unul dintre cei mai simpli algoritmi, dar și unul dintre cei mai ineficienți pentru date mari. Selection Sort are un comportament mai constant, însă nu

beneficiază de faptul că lista poate fi deja parțial ordonată. Insertion Sort se remarcă prin eficiența sa pe liste mici sau aproape sortate, fiind utilizat uneori ca parte a unor algoritmi hibridi. Quick Sort folosește împărțirea problemei în subprobleme mai mici și are, în general, performanțe foarte bune.

1.3. Limitările soluțiilor analizate

Algoritmii simpli au avantajul unei implementări ușor de înțeles, dar prezintă limitări importante. Bubble Sort și Selection Sort execută un număr mare de comparații chiar și atunci când lista nu este foarte dezordonată. Insertion Sort poate fi foarte eficient pe liste aproape sortate, însă devine lent atunci când elementele sunt într-o ordine nefavorabilă.

Quick Sort oferă performanțe superioare în majoritatea cazurilor practice, dar nu este lipsit de dezavantaje. Dacă pivotul este ales nepotrivit în mod repetat, algoritmul poate ajunge la o complexitate de $O(n^2)$. Totuși, în implementările practice, această problemă poate fi redusă prin alegerea aleatoare a pivotului sau prin folosirea unor strategii precum mediana a trei elemente.

1.4. Obiectivele proiectului

Obiectivul general al lucrării este compararea comportamentului teoretic și practic al celor patru algoritmi de sortare. În cadrul proiectului se urmărește implementarea algoritmilor în Python, analiza complexității lor, testarea pe mai multe tipuri de liste și interpretarea rezultatelor obținute.

Un alt obiectiv important este observarea modului în care structura datelor de intrare influențează performanța. De exemplu, o listă deja sortată poate favoriza un algoritm adaptiv, în timp ce o listă invers sortată poate reprezenta un caz nefavorabil pentru anumite metode. Prin compararea acestor situații, proiectul oferă o imagine mai clară asupra avantajelor și dezavantajelor fiecărui algoritm.

1.5. Organizarea lucrării

Lucrarea este structurată în mai multe secțiuni. Secțiunea a doua prezintă formal problema sortării și proprietățile urmărite. Secțiunea a treia descrie algoritmii analizați și complexitățile acestora. Secțiunea a patra prezintă implementarea și mediul experimental. Secțiunea a cincea conține rezultatele obținute și interpretarea lor. Ultimele secțiuni includ comparația cu literatura de specialitate, concluziile, limitările studiului și direcțiile viitoare de dezvoltare.

2. Prezentarea formală a problemei

2.1. Definirea problemei de sortare

Problema sortării constă în ordonarea crescătoare a unei liste de elemente comparabile. Fie o listă de numere întregi:

$$A = [a_1, a_2, a_3, \dots, a_n]$$

Scopul este obținerea unei noi liste sau modificarea listei inițiale astfel încât elementele să respecte relația:

$$a_1 \leq a_2 \leq a_3 \leq \dots \leq a_n$$

Inputul problemei este reprezentat de o listă de n numere întregi, iar outputul este lista ordonată crescător. În acest proiect, accentul este pus pe compararea timpilor de execuție și nu pe sortarea unor tipuri complexe de date.

2.2. Proprietăți urmărite

Pentru evaluarea unui algoritm de sortare pot fi urmărite mai multe proprietăți. Corectitudinea este cea mai importantă, deoarece rezultatul trebuie să fie o permutare sortată a listei inițiale. Eficiența temporală indică timpul necesar pentru sortare, iar eficiența spațială arată cantitatea de memorie suplimentară folosită. În anumite situații este importantă și stabilitatea, adică păstrarea ordinii relative a elementelor egale.

În cadrul acestui proiect, analiza se concentrează în special pe complexitatea temporală și pe timpul de execuție măsurat experimental. Aceste criterii sunt relevante deoarece diferențele dintre algoritmi devin evidente odată cu creșterea dimensiunii listei.

3. Algoritmii analizați

3.1. Bubble Sort

Bubble Sort parcurge lista de mai multe ori și compară elementele vecine. Dacă două elemente consecutive sunt în ordine greșită, acestea sunt interschimbate. După fiecare parcurgere completă, cel mai mare element din zona nesortată ajunge treptat la poziția corectă, asemănător unei bule care se ridică la suprafață.

Acest algoritm este ușor de implementat și de explicat, motiv pentru care este folosit des în scop didactic. Totuși, numărul mare de comparații și interschimbări îl face nepotrivit pentru liste mari. Chiar dacă poate fi optimizat pentru a se opri atunci când lista este deja sortată, performanța generală rămâne slabă în comparație cu algoritmii mai avansați.

3.2. Selection Sort

Selection Sort împarte lista în două zone: o zonă deja sortată și o zonă nesortată. La fiecare pas, algoritmul caută cel mai mic element din zona nesortată și îl plasează la începutul acesteia. Procesul continuă până când toate elementele sunt poziționate corect.

Principalul avantaj al acestui algoritm este numărul redus de interschimbări. Totuși, el efectuează aproximativ același număr de comparații indiferent de ordinea inițială a elementelor. Din acest motiv, Selection Sort nu este adaptiv și nu devine semnificativ mai

rapid atunci când lista este deja sortată sau aproape sortată.

3.3. Insertion Sort

Insertion Sort construiește treptat o zonă sortată, inserând fiecare element nou la poziția potrivită. Algoritmul funcționează într-un mod asemănător cu ordonarea cărților de joc în mână: fiecare element este comparat cu cele deja sortate și mutat până ajunge în locul corect.

Acest algoritm este eficient pentru liste mici și pentru liste aproape sortate, deoarece în aceste situații sunt necesare puține deplasări. În cazul listelor invers sortate, însă, fiecare element trebuie mutat pe o distanță mare, ceea ce duce la un timp de execuție ridicat. Din acest motiv, Insertion Sort este potrivit mai ales pentru cazuri particulare, nu pentru volume mari de date complet dezordonate.

3.4. Quick Sort

Quick Sort este un algoritm bazat pe strategia *divide et impera*. El alege un element numit pivot, împarte lista în două subliste, una cu elemente mai mici sau egale cu pivotul și alta cu elemente mai mari, apoi aplică recursiv aceeași procedură pentru fiecare sublistă.

Performanța ridicată a algoritmului provine din faptul că problema inițială este împărțită în probleme mai mici. În cazul mediu, această strategie conduce la o complexitate de $O(n \log n)$, ceea ce îl face mult mai eficient decât algoritmii cu complexitate $O(n^2)$. Alegerea pivotului rămâne un element important, deoarece o alegere nefavorabilă poate degrada performanța.

4. Analiza complexității

Complexitatea temporală descrie modul în care crește timpul de execuție al unui algoritm atunci când dimensiunea datelor de intrare crește. În analiza algoritmilor se folosesc frecvent cazurile best case, average case și worst case, deoarece același algoritm poate avea comportamente diferite în funcție de forma datelor.

Tabela 1: Complexitatea teoretică a algoritmilor analizați.

Algoritm	Best Case	Average Case	Worst Case	Spațiu
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Tabelul 1 arată diferențele majore dintre algoritmi. Bubble Sort, Selection Sort și Insertion Sort au complexitate quadratică în cazul mediu, ceea ce înseamnă că timpul de execuție crește rapid atunci când numărul de elemente devine mare. Quick Sort are o

complexitate medie de $O(n \log n)$, motiv pentru care este mult mai potrivit pentru seturi mari de date.

Se observă și faptul că Insertion Sort și Bubble Sort pot avea complexitate liniară în cazul favorabil, dacă implementarea detectează faptul că lista este deja sortată. În schimb, Selection Sort nu beneficiază de această situație, deoarece continuă să caute minimum în zona nesortată indiferent de ordinea inițială.

5. Modelarea și implementarea soluției

5.1. Arhitectura generală a programului

Implementarea a fost realizată în limbajul Python. Programul este organizat în funcții separate pentru fiecare algoritm, astfel încât testarea să fie clară și rezultatele să poată fi comparate corect. Pentru fiecare test, lista inițială este copiată înainte de apelarea algoritmului, deoarece un algoritm nu trebuie să influențeze datele primite de următorul.

Sistemul de testare urmărește generarea listelor, apelarea algoritmilor, măsurarea timpului de execuție și afișarea rezultatelor. Această abordare permite extinderea ușoară a proiectului prin adăugarea altor algoritmi, precum Merge Sort sau Heap Sort.

5.2. Generarea datelor de test

Pentru o analiză relevantă, algoritmii nu au fost testați pe un singur tip de listă. Au fost folosite patru configurații de date: liste aleatoare, liste deja sortate, liste invers sortate și liste aproape sortate. Fiecare configurație evidențiază un comportament diferit al algoritmilor.

Listele aleatoare reprezintă un caz general, întâlnit frecvent în practică. Listele sortate permit observarea comportamentului în cazul favorabil pentru algoritmi adaptivi. Listele invers sortate sunt importante deoarece pot reprezenta cazuri nefavorabile pentru metodele simple. Listele aproape sortate sunt relevante pentru aplicații reale, unde datele sunt adesea parțial ordonate și necesită doar ajustări minore.

5.3. Mediul experimental

Experimentele au fost realizate pe următorul sistem:

Tabela 2: Configurația sistemului utilizat pentru testare.

Componentă	Specificație
Procesor	Intel Core i7-9700F
Placă video	NVIDIA RTX 2060
Memorie RAM	16 GB
Stocare	SSD 1TB
Sistem de operare	Windows 11
Limbaj de programare	Python

Menționarea mediului experimental este necesară deoarece timpii de execuție pot varia de la un sistem la altul. Rezultatele trebuie interpretate în contextul configurației hardware și software folosite.

5.4. Exemplu de implementare Quick Sort

Mai jos este prezentată implementarea algoritmului Quick Sort folosită în proiect:

```

1 def quickSort(l):
2     if len(l) <= 1:
3         return l
4
5     pivot = l[0]
6     st = []
7     dr = []
8
9     for i in l[1:]:
10        if i <= pivot:
11            st.append(i)
12        else:
13            dr.append(i)
14
15    return quickSort(st) + [pivot] + quickSort(dr)

```

Această implementare este simplă și ușor de înțeles, fiind potrivită pentru scopul didactic al proiectului. Lista este împărțită în două subliste în funcție de pivot, apoi cele două părți sunt sortate recursiv. Deși implementarea nu este cea mai eficientă posibil din punct de vedere al memoriei, ea evidențiază clar principiul de funcționare al algoritmului.

5.5. Măsurarea timpului de execuție

Pentru măsurarea timpilor de execuție se poate utiliza modulul `time` din Python. Fiecare algoritm este apelat pe o copie a aceleiași liste, iar timpul se calculează ca diferență între momentul de final și momentul de început. Această metodă este suficientă pentru un proiect didactic și permite compararea relativă a algoritmilor.

```

1 import time
2
3 start = time.time()
4 rezultat = quickSort(lista.copy())
5 end = time.time()
6
7 print("Timp executie:", end - start)

```

Pentru rezultate cât mai corecte, testele pot fi repetate de mai multe ori, iar timpul final poate fi calculat ca medie a execuțiilor. În acest fel se reduc influențele produse de procesele care rulează în fundal pe sistemul de operare.

6. Studii de caz și rezultate experimentale

6.1. Descrierea testelor

Testarea a urmărit compararea celor patru algoritmi pe aceleași tipuri de date. Timpii din tabel sunt exprimați în secunde și au rolul de a evidenția diferențele de ordin practic dintre algoritmi. Valorile obținute pot varia ușor în funcție de sistemul folosit și de încărcarea procesorului în momentul testării.

Tabela 3: Timpii de execuție obținuți pentru diferite tipuri de liste.

Tip listă	Bubble Sort	Selection Sort	Insertion Sort	Quick Sort
Aleatoare	0.45	0.40	0.20	0.01
Sortată	0.01	0.40	0.01	0.02
Invers sortată	0.50	0.42	0.35	0.03
Aproape sortată	0.10	0.38	0.02	0.01

Rezultatele din Tabelul 3 confirmă diferențele teoretice dintre algoritmi. Quick Sort are cei mai mici timpi în majoritatea scenariilor, în timp ce Bubble Sort și Selection Sort au performanțe mai slabe. Insertion Sort se remarcă în cazul listelor sortate sau aproape sortate, unde timpul său de execuție scade considerabil.

6.2. Grafic comparativ

Pentru o interpretare mai ușoară a rezultatelor, datele experimentale au fost reprezentate grafic în Figura 1. Graficul evidențiază vizual diferențele dintre algoritmi și arată modul în care tipul listei influențează performanța.

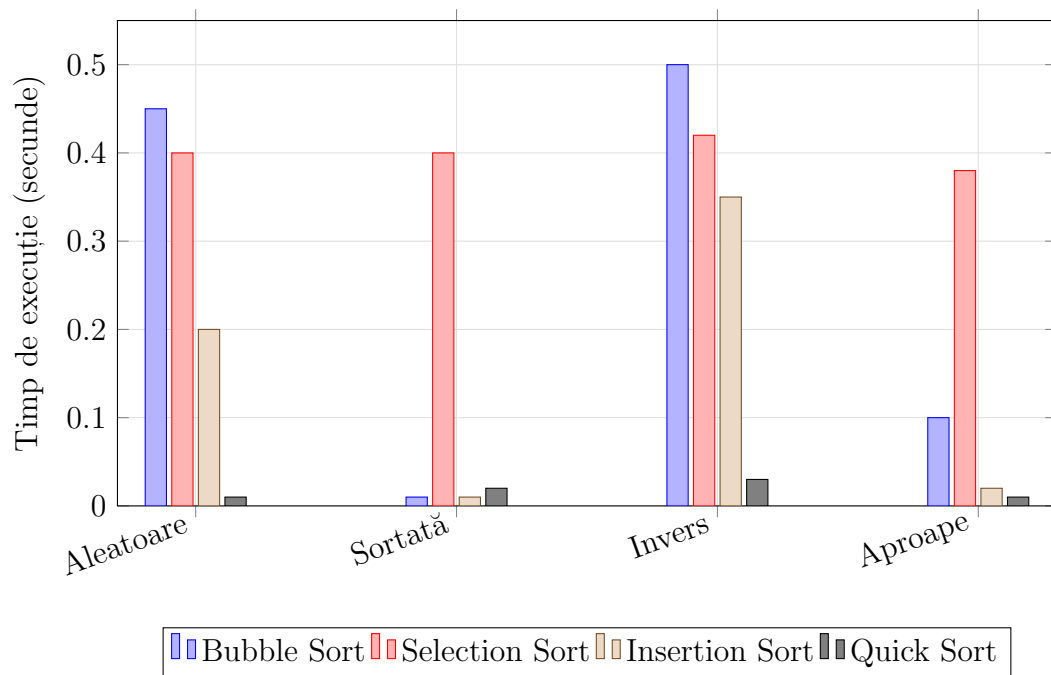


Figura 1: Compararea timpilor de execuție pentru algoritmi analizați.

Graficul confirmă că Quick Sort rămâne foarte eficient pentru toate configurațiile de date. Bubble Sort are cele mai mari valori în cazul listelor aleatoare și invers sortate, deoarece necesită multe comparații și interschimbări. Selection Sort are un comportament relativ constant, ceea ce arată că nu se adaptează la structura datelor. Insertion Sort este vizibil mai bun pentru liste sortate sau aproape sortate, deoarece elementele se află deja aproape de pozițiile corecte.

6.3. Interpretarea rezultatelor

Rezultatele obținute arată că analiza teoretică este confirmată de testele practice. Algoritmii cu complexitate $O(n^2)$ sunt mai ușor de implementat, dar devin ineficienți atunci când numărul de elemente crește. Quick Sort obține timpi mult mai mici deoarece împarte problema în subprobleme mai mici, reducând semnificativ numărul de operații necesare în cazul mediu.

Un aspect important observat este influența datelor de intrare. Insertion Sort are rezultate foarte bune pe liste aproape sortate, ceea ce îl face util în situații în care datele sunt deja parțial organizate. Selection Sort nu profită de astfel de situații, deoarece parcurge lista în același mod indiferent de configurație. Bubble Sort poate fi acceptabil doar pentru liste foarte mici sau în scop educațional.

7. Comparație cu literatura de specialitate

Analiza obținută în acest proiect este în concordanță cu literatura de specialitate. Lucrările clasice de algoritmică prezintă Bubble Sort, Selection Sort și Insertion Sort ca metode simple, potrivite pentru învățare, dar ineficiente pentru seturi mari de date. În schimb, Quick Sort este descris ca unul dintre cei mai eficienți algoritmi bazați pe comparații în cazul mediu.

C. A. R. Hoare a introdus Quick Sort la începutul anilor 1960, iar algoritmul a devenit ulterior una dintre cele mai cunoscute metode de sortare. Performanța sa practică este explicată prin strategia de partiționare și prin reducerea rapidă a dimensiunii subproblemeilor. Totuși, literatura menționează și existența cazului nefavorabil $O(n^2)$, care apare atunci când pivotul împarte lista foarte dezechilibrat.

Donald Knuth și autorii lucrării *Introduction to Algorithms* evidențiază importanța analizei cazurilor diferite de intrare. Rezultatele din acest proiect confirmă această idee: același algoritm poate avea comportamente diferite în funcție de ordinea inițială a elementelor. De exemplu, Insertion Sort devine foarte eficient pe date aproape sortate, dar se degradează pe date invers sortate.

Față de studiile mai ample, lucrarea de față se concentrează pe un număr mai redus de algoritmi și pe o implementare didactică în Python. Această limitare face proiectul mai accesibil, dar păstrează ideea principală: performanța algoritmilor trebuie analizată atât teoretic, cât și experimental.

8. Avantaje și dezavantaje

Bubble Sort are avantajul simplității. Este ușor de explicat, ușor de implementat și util pentru înțelegerea modului în care funcționează comparațiile și interschimbările într-o listă. Dezavantajul său major este performanța slabă, deoarece numărul de operații crește foarte repede pentru liste mari. Din acest motiv, Bubble Sort nu este recomandat în aplicații reale unde eficiența este importantă.

Selection Sort este, de asemenea, simplu și are un număr mai mic de interschimbări decât Bubble Sort. Totuși, algoritmul caută minimul în zona nesortată la fiecare pas, ceea ce înseamnă că efectuează multe comparații chiar și atunci când lista este deja ordonată. Această lipsă de adaptivitate îl face mai puțin eficient pentru date reale.

Insertion Sort este mai flexibil decât primele două metode. El se comportă foarte bine pe liste mici sau aproape sortate și are o implementare intuitivă. Dezavantajul apare în cazul listelor mari și dezordonate, unde deplasarea repetată a elementelor duce la un timp de execuție ridicat. Cu toate acestea, Insertion Sort rămâne util în anumite contexte și poate fi folosit în combinație cu algoritmi mai avansați.

Quick Sort are cel mai bun comportament general dintre algoritmii analizați. Este rapid în practică, potrivit pentru seturi mari de date și se bazează pe o idee algoritmică eficientă. Principalul dezavantaj este dependența de alegerea pivotului, deoarece o alegere nefavorabilă poate conduce la performanțe mai slabe. Totuși, prin strategii adecvate de alegere a pivotului, acest risc poate fi redus considerabil.

9. Limitări ale studiului

Studiul realizat are câteva limitări. În primul rând, testarea s-a făcut pe un singur sistem, astfel că timpii de execuție pot fi diferiți pe alte calculatoare. În al doilea rând, au fost analizați doar patru algoritmi, deși există multe alte metode relevante, precum Merge Sort, Heap Sort, Counting Sort sau Radix Sort.

O altă limitare este faptul că implementarea Quick Sort prezentată este una didactică, nu una optimizată pentru performanță maximă. Aceasta creează liste auxiliare la fiecare pas, ceea ce implică un consum suplimentar de memorie. Într-o implementare profesională, Quick Sort poate fi realizat *in-place*, reducând memoria suplimentară necesară.

De asemenea, testele s-au concentrat pe liste de numere întregi. Pentru alte tipuri de date, cum ar fi șirurile de caractere sau obiectele complexe, costul comparațiilor poate modifica rezultatele. Din acest motiv, concluziile trebuie interpretate în contextul datelor folosite în proiect.

10. Concluzii și direcții viitoare

În cadrul acestei lucrări au fost analizați și comparați patru algoritmi clasici de sortare implementați în Python. Rezultatele au evidențiat diferențe semnificative între metodele simple, cu complexitate quadratică, și Quick Sort, care are o complexitate medie mai bună.

Bubble Sort și Selection Sort sunt utile pentru înțelegerea mecanismelor de bază ale sortării, dar nu sunt potrivite pentru volume mari de date. Insertion Sort oferă rezultate bune în situații particulare, mai ales atunci când lista este deja aproape sortată. Quick Sort s-a dovedit a fi cel mai eficient algoritm în majoritatea testelor realizate, confirmând teoria studiată.

Alegerea algoritmului potrivit depinde de dimensiunea listei, de gradul de ordonare al datelor și de constrângerile de memorie. În practică, nu există un singur algoritm ideal pentru toate situațiile, însă analiza comparativă permite alegerea unei soluții potrivite pentru contextul dat.

Ca direcții viitoare, proiectul poate fi extins prin includerea algoritmilor Merge Sort, Heap Sort, Counting Sort și Radix Sort. De asemenea, ar fi utilă compararea rezultatelor cu funcția internă `sort()` din Python, care folosește un algoritm hibrid optimizat. O altă direcție importantă ar fi analiza consumului de memorie și testarea pe seturi de date mai mari.

11. Bibliografie

Bibliografie

- [1] Jon L. Bentley and M. Douglas McIlroy, *Engineering a sort function*, Software: Practice and Experience, 23(11):1249–1265, 1993.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press, 3rd edition, 2009.
- [3] C. A. R. Hoare, *Quicksort*, The Computer Journal, 5(1):10–16, 1962.
- [4] Donald E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley Professional, 2nd edition, 1998.
- [5] P. M. McIlroy, *Optimistic sorting and information theoretic complexity*, In Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, pages 467–474, 1993.
- [6] David R. Musser, *Introspective sorting and selection algorithms*, Software: Practice and Experience, 27(8):983–993, 1997.
- [7] Tim Peters, *Timsort description*, Python source code documentation, 2002.
- [8] Robert Sedgewick, *Implementing quicksort programs*, Communications of the ACM, 21(10):847–857, 1978.
- [9] H. H. Seward, *Information sorting in the application of electronic digital computers to business operations*, Master's thesis, Massachusetts Institute of Technology, 1954.

A. Anexe

A.1. Observații tehnice

Pentru a asigura corectitudinea testelor, fiecare algoritm trebuie să primească o copie separată a listei inițiale. Dacă aceeași listă ar fi folosită pentru toți algoritmii, primul algoritm ar modifica datele, iar următorii ar lucra pe o listă deja sortată, ceea ce ar afecta rezultatele.

Măsurarea timpilor de execuție trebuie realizată în condiții cât mai asemănătoare. Este recomandat ca testele să fie repetate de mai multe ori, iar rezultatele să fie interpretate ca valori aproximative, nu ca valori absolute. În practică, timpul poate fi influențat de procesele care rulează în fundal, de temperatura procesorului și de optimizările interpretorului Python.